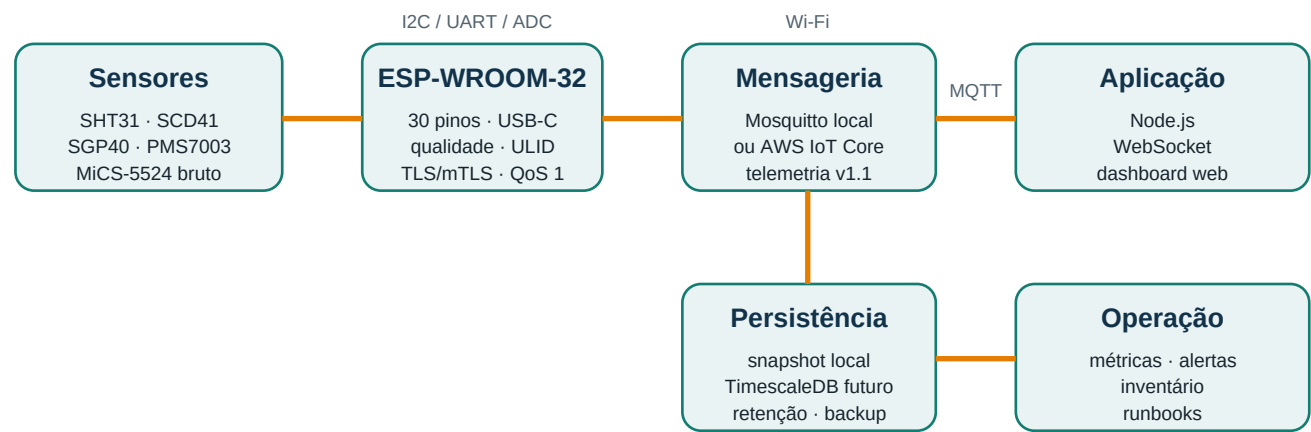


ARQUITETURA DA SOLUÇÃO

Monitor IoT de qualidade do ar · versão final do protótipo



Objetivo

Plataforma educacional para observar temperatura, umidade, CO2, material particulado e um índice de VOC. O projeto integra ESP-WROOM-32 DevKit 30P/USB-C, MQTT, backend/dashboard e uma trilha executável de evolução para AWS.

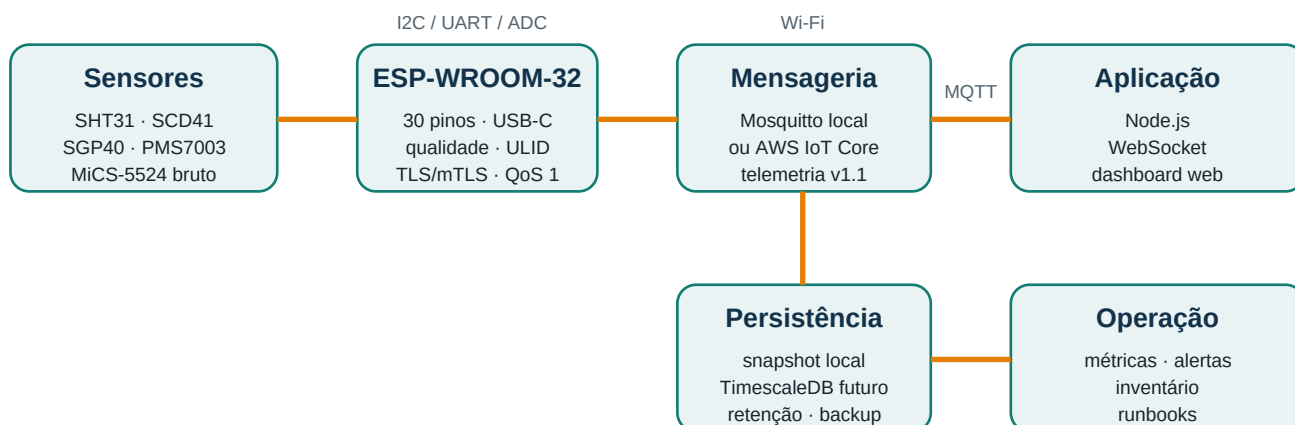
ESCOPO DE SEGURANÇA

Não é instrumento certificado, alarme de incêndio, detector de vazamento nem equipamento médico. Leituras exigem caracterização, calibração e comparação com referência antes de qualquer decisão crítica.

Estado desta entrega

Componente	Estado verificado
Firmware HIL/físico/AWS	Três builds; maior imagem usa 60,7% do slot OTA
Placa alvo	ESP-WROOM-32 DevKit 30P/USB-C; GPIOs cruzados
Contrato	13 testes Python, 9 Node e JSON Schema v1.1
Stack local	Compose, MQTT, persistência, API, PWA e mobile testados
AWS	IaC e pacote Lambda testáveis; conta real não foi alterada
CAD/EDA	Roadmaps completos; arquivos nativos dependem das ferramentas e medições

1. Visão funcional



Fluxo principal

- O ESP32 amostra barramentos I2C/UART e a entrada analógica em cadências independentes.
- Cada publicação consolida a leitura mais recente, marca validade/qualidade e carrega identidade de boot.
- O broker autentica o dispositivo e distribui mensagens; o backend rejeita tópico divergente, duplicidade e payload fora do contrato.
- O dashboard recebe estado e séries curtas por WebSocket; snapshots atômicos restauram a instalação local.

Princípios adotados

Princípio	Aplicação
Contrato antes da interface	Schema e validadores compartilhando limites e semântica
Falha explícita	Campo nulo + quality/status; não fabricar medida
Segredo fora do código	secrets.h e .env ignorados; certificados provisionados
Evolução mensurável	gates de teste e critérios de aceite por etapa

Topologia MQTT

Publicação: **qualidade-ar/{site_id}/{device_id}/telemetria**. Estado/LWT:

qualidade-ar/{site_id}/{device_id}/status. No IoT Core, Thing Name, client ID e device_id devem coincidir; a política limita a publicação ao site/dispositivo.

2. Nó de sensoriamento

Elemento	Interface	Saída no protótipo	Tratamento
SHT31	I2C	temperatura/umidade	CRC no driver, faixa e stale
SCD41	I2C	CO2/temperatura/umidade	data-ready e aquecimento
SGP40	I2C	VOC index	compensação T/RH + Gas Index
PMS7003	UART	PM1/PM2.5/PM10	frame 32 B + checksum
MiCS-5524	ADC	tensão bruta	média, divisor e caracterização futura

Cadência e temporalidade

A aquisição não deve bloquear o laço principal. O SGP40 precisa de amostragem periódica para estabilizar o algoritmo; o SCD41 publica quando há dado pronto; o PMS7003 é analisado por máquina de estados. A mensagem só é emitida após o relógio NTP produzir timestamp plausível.

Interpretação

- CO2 do SCD41 é uma medida física; respeitar tempo de aquecimento, instalação e condições ambientais.
- VOC index é adimensional e relativo ao histórico do sensor; não equivale a ppm de um gás específico.
- O divisor resistivo do MiCS-5524 protege/condiciona o ADC, mas não transforma tensão em ppm. Publicar apenas o sinal bruto até existir curva validada.

Pinagem, alimentação e integridade

GPIO21/22 atendem I2C, GPIO16/17 a UART2 e GPIO34/ADC1 o canal analógico. GPIO0/2/5/12/15 não são usados porque afetam o boot. No modo USB, JP1 fica aberto; no modo autônomo, USB desconectado e JP1 fechado. A entrada externa é 5 V/2 A, com LDO 3,3 V dedicado. O segundo rótulo VIN na posição de VN/GPIO39 permanece bloqueado até conferência física.

Memória do módulo

A tabela para flash de 4 MB reserva dois slots OTA de 1.572.864 bytes e LittleFS. O payload/buffer MQTT é limitado a 896 bytes e a publicação exige pelo menos 30 kB de heap livre; TLS precisa de soak test com os certificados PEM reais.

3. Contrato de telemetria v1.1

Grupo	Campos essenciais
Identidade	schema_version, message_id ULID, site_id, device_id, boot_id, sequence
Tempo	sent_at RFC 3339 com fuso
Medidas	temperature_c, humidity_pct, co2_ppm, voc_index, pm1/pm25/pm10, lpg_ppm e gas_raw_v
Qualidade	gas_status e sensor_status
Metadados	firmware, placa/revisão, modo, RSSI, uptime e heap

Regras de ingestão

- Rejeitar versão desconhecida, número não finito, sequência inválida, identidade fora do padrão ou tópico incompatível.
- Aceitar nulo somente quando a qualidade indicar ausência/erro; status OK não pode esconder medida obrigatória nula.
- Deduplicar por message_id com janela limitada; usar boot_id para distinguir reinício de reordenação.
- Aplicar limites de corpo, quantidade de dispositivos e tamanho de séries antes de alocar memória.

Exemplo reduzido

```
{
  "schema_version": "1.1",
  "message_id": "01J...", "sequence": 42,
  "sent_at": "2026-07-31T15:00:00Z",
  "site_id": "lab-ufg", "device_id": "qar-001",
  "measurements": {"co2_ppm": 612, "voc_index": 103},
  "quality": {"gas_status": "SAFE", "sensor_status": "OK"}
}
```

4. Backend, dashboard e persistência

Base local atual

O serviço Node.js consome MQTT ou aceita HTTP apenas quando habilitado, valida o contrato, mantém séries limitadas, salva snapshots JSON atômicos e transmite atualizações por WebSocket. A PWA mostra cartões, histórico curto, conectividade e avisos de caráter experimental.

Controle	Implementado / direção
Entrada	Content-Type, body limit, token opcional, validação de tópico/payload
Memória	mapas limitados, deduplicação LRU e quantidade máxima por série
Persistência	snapshot versionado, gravação atômica, volume e restauração testada
Web	CSP, headers, PWA e caminho estático resolvido com segurança
Transporte	MQTT QoS 1; TLS no ambiente de produção
Histórico longo	TimescaleDB após definição de retenção e migrações

Modelo recomendado

- telemetry_raw: evento validado, chave de tempo/dispositivo e payload rastreável.
- device_registry: identidade, local, versão e estado de provisionamento.
- telemetry_5m/1h: agregados contínuos para visualização e custo controlado.
- quality_events: transições de erro, ausência e recuperação para operação.

Capacidade

A planilha **estimativa_carga.xlsx** calcula mensagens/s, pico, volume diário/mensal e headroom. Os números são premissas: medir payload, throughput, índices, WAL, compressão e retenção no ambiente escolhido antes de dimensionar produção.

5. Segurança e operação

Camada	Controles mínimos
Dispositivo	credencial única, secure boot/flash encryption quando aplicável, OTA assinado
Rede	Wi-Fi segregado, egress mínimo, NTP confiável
Broker	TLS, ACL por identidade, sem anônimo, limites de conexão/mensagem
Aplicação	segredo por ambiente, dependências fixadas, headers, limites e logs sanitizados
Dados	retenção definida, backup testado, acesso mínimo e trilha de auditoria
Operação	métricas, alertas, runbook, inventário e rotação de credenciais

Observabilidade

- Taxa de mensagens recebidas/rejeitadas/duplicadas e latência fim a fim.
- Dispositivos online, último timestamp, boot_id, RSSI e versão de firmware.
- Percentual de medidas válidas por sensor e tempo em estado de erro/stale.
- Uso de CPU/memória/disco, conexões MQTT/WS e crescimento do banco.

Resposta a falhas

Os runbooks devem separar perda de energia, Wi-Fi, DNS/NTP, TLS, ACL, sensor ausente, leitura fora de faixa e saturação de infraestrutura. Cada incidente precisa de evidência reproduzível e critério explícito de retorno ao serviço.

6. Hardware, PCB e invólucro

EasyEDA Pro

O roadmap de hardware parte de requisitos e arquitetura de alimentação, converte a netlist funcional em esquemático hierárquico, associa footprints conferidos em escala 1:1, atualiza a PCB, define regras, posiciona por blocos e executa ERC/DRC antes de Gerbers e montagem.

- Revisar datasheet, pinagem, tensão, corrente, footprint, orientação e disponibilidade de cada item da BOM.
- Manter I2C curto com pull-ups calculados; afastar ADC de retornos de corrente e antena; respeitar keepout do ESP32.
- Adicionar test points em trilhos, terra, UART, I2C e entrada analógica.
- Fazer bring-up por estágios: inspeção, resistência, fonte limitada, trilhos, programação e sensores um a um.

SolidWorks

O roadmap mecânico começa por um envelope medido da placa montada, usa variáveis globais, modela base/tampa parametrizadas, posiciona torres e recortes em contexto e valida ventilação, montagem, tolerâncias, interferências e impressão.

- A entrada de ar não deve receber jato direto nem formar volume morto; separar termicamente regulador, ESP32 e sensores.
- Proteger o PMS7003 contra recirculação entre entrada e saída.
- Usar protótipos de ajuste antes do STL final; registrar material, orientação, altura de camada e compensações.

Entregáveis de fabricação

PCB	Mecânica
.epro/.json do projeto; PDF do esquemático	SLDPRT/SLDASM com parâmetros
Gerber + drill + pick-and-place	STEP de intercâmbio e desenhos cotados
BOM revisada e relatório DRC	STL/3MF testado e registro de revisão

7. AWS IoT Core e operação de frota

Caminho direto

O perfil esp32-aws usa endpoint ATS, porta 8883, CA, certificado e chave exclusivos. Thing Name, MQTT client ID e device_id coincidem; o atributo siteld restringe o tópico na política. TLS/mTLS está compilado, mas só o sandbox com certificado real comprova handshake, heap e política.

Componente	Responsabilidade
IoT Core	autenticação mTLS, MQTT QoS 1 e regra por tópico
SQS Standard + DLQ	desacoplamento, reentrega e quarentena; consumidor idempotente
S3 bruto	evento original criptografado, privado e com lifecycle
Lambda de referência	validação do contrato/tópico e falha parcial por lote
DynamoDB	estado atual condicionado por ULID
CloudWatch	alarme de DLQ; ampliar para custo, falhas e latência

Alternativa de borda

Um gateway Mosquitto pode concentrar fila offline e a conexão mTLS, reduzindo pressão de memória nos nós. Em troca, ele se torna componente crítico e precisa de disco, observabilidade, backup, atualização e certificado próprio.

Provisionamento e OTA

- Um certificado por dispositivo; testar emissão, rotação e revogação antes da frota.
- Os dois slots OTA permitem rollback, mas download, assinatura, confirmação da imagem e orquestração por Jobs ainda devem ser implementados.
- Aplicar o CloudFormation apenas em conta sandbox com orçamento, revisão do change set e autorização explícita.

8. Verificação e critérios de aceite

Gate	Evidência exigida
G0 - contrato	testes Python e JavaScript; amostras válidas/inválidas
G1 - software	Compose, persistência, HTTP/WS/PWA, limites e reconexão
G2 - firmware	build HIL/físico/AWS, serial, heap TLS e logs
G3 - elétrica	pinout/flash confirmados, ERC/DRC, trilhos e consumo
G4 - sensores	coleta lado a lado, repetibilidade, stale/erro e referência
G5 - AWS	mTLS, menor privilégio, reentrega/DLQ e revogação
G6 - sistema	ensaio prolongado, recuperação, deduplicação e retenção
G7 - entrega	documentação, BOM/CAD/EDA, versões e riscos

O que já foi automatizado

- 13 testes Python, 9 testes Node.js e 10 testes Flutter.
- Autoteste do simulador, análise Flutter e compilação do código Python/JavaScript.
- Build PlatformIO dos ambientes esp32-hil, esp32-físico e esp32-aws.
- Compose com MQTT, Web/PWA, ingestão, WebSocket e restauração após reinício.

O que depende de laboratório

- Validação elétrica, calibração/caracterização, comparação contra instrumentos e ensaio térmico.
- Criação e revisão nativa do projeto EasyEDA Pro.
- Modelagem, interferência e prototipagem nativa no SolidWorks.

Critério de encerramento

A versão só deve ser promovida quando todos os gates aplicáveis tiverem evidências anexadas, riscos residuais aceitos e instruções de reprodução atualizadas. Aprovação acadêmica não converte o protótipo em equipamento certificado.

9. Roadmap integrado

Fase	Resultado	Saída de controle
A - congelar requisitos	escopo, contrato, placa/pinout	baseline v1.1
B - bancada	sensores reais e telemetria estável	logs + relatório de teste
C - PCB rev. A	placa fabricável e montada	Gerber/BOM/DRC/bring-up
D - case rev. A	fluxo de ar e montagem validados	CAD/STEP/STL + ensaio
E - dados	persistência, retenção e dashboards	migrações + benchmark
F - sandbox AWS	mTLS, regra, DLQ e estado	evidências + custo
G - operação	provisionamento, OTA, métricas e runbooks	checklist de liberação
H - piloto	uso controlado, análise e correções	relatório e decisão go/no-go

Documentos de execução

- ROADMAP_GERAL.md - sequência, gates e definição de pronto.
- ROADMAP_FIRMWARE.md e ROADMAP_SOFTWARE.md - evolução por componente.
- ROADMAP_HARDWARE_EASYEDA.md - esquemático, PCB, fabricação e bring-up.
- ROADMAP_CASE_SOLIDWORKS.md - CAD paramétrico, ventilação e fabricação.
- PLANO_TESTES.md - matriz de verificação do código à bancada.

Referências normativas e técnicas

Usar as folhas de dados e bibliotecas oficiais dos fabricantes, documentação oficial do ESP-IDF/Arduino-ESP32, MQTT 3.1.1/5.0 conforme a implantação e as recomendações de qualidade do ar apenas como contexto de comunicação. O limite operacional do sistema deve nascer da aplicação e de validação metrológica, não de uma cor arbitrária no dashboard.

Documento gerado em 2 de agosto de 2026 · Consulte o repositório para código e revisões posteriores.